

Computer Vision Practical Report:

VIDEO DETECTION

1. AIM

To detect and localize vehicles in a video frame sequence using a pre-trained Haar Cascade Classifier (cars.xml) or Background Subtraction with contour analysis in OpenCV, draw bounding boxes around identified vehicles, display the real-time detections, and save the output.

2. PROCEDURE

- Load the target video file using `cv2.VideoCapture()`.
- Load the pre-trained vehicle detection model using `cv2.CascadeClassifier('cars.xml')` (or initialize a background subtractor like `cv2.createBackgroundSubtractorMOG2()`).
- Read frame-by-frame inside a continuous while loop using `cap.read()`.
- Convert each frame to grayscale (`cv2.cvtColor()`) and apply Gaussian blurring (`cv2.GaussianBlur()`) to reduce background noise.
- Pass the grayscale frame into `detectMultiScale()` to extract bounding coordinates (x, y, w, h) for detected vehicles (or extract foreground masks and contours using `cv2.findContours()`).
- Draw bounding rectangles around each detected vehicle using `cv2.rectangle()`.
- Render the processed output frames in real time using `cv2.imshow()`, write frames to disk using `cv2.VideoWriter()`, and release resources (`cap.release()`, `cv2.destroyAllWindows()`) when finished.

3. PROGRAM CODE

```
import cv2
import matplotlib.pyplot as plt

# 1. Open video file
video_path = 'E39-VID.mp4' # Replace with your video file path
cap = cv2.VideoCapture(video_path)
```

2. Create Background Subtractor

```
object_detector = cv2.createBackgroundSubtractorMOG2(history=100, varThreshold=40)
```

```
while cap.isOpened():
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

3. Create a foreground mask for moving objects

```
mask = object_detector.apply(frame)
```

```
_, mask = cv2.threshold(mask, 254, 255, cv2.THRESH_BINARY) # Remove shadows
```

4. Find contours of moving vehicles

```
contours, _ = cv2.findContours(mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
```

```
output_frame = frame.copy()
```

```
for cnt in contours:
```

```
    # Filter out small contours (noise/pedestrians) by area threshold
```

```
    area = cv2.contourArea(cnt)
```

```
    if area > 800:
```

```
        x, y, w, h = cv2.boundingRect(cnt)
```

```
        cv2.rectangle(output_frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

```
        cv2.putText(output_frame, "Vehicle", (x, y - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0),
```

2)

5. Display Input Frame and Output Frame side-by-side on the same window

```
combined = cv2.hconcat([frame, output_frame])
```

```
cv2.imshow("Vehicle Detection - Left: Original | Right: Detected", combined)
```

```
# Press 'q' on keyboard to exit
```

```
if cv2.waitKey(30) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()  
cv2.destroyAllWindows()
```

4. OUTPUT



5. RESULT

The Python script executed successfully. The input image was loaded.